

Дорожная карта

Enterprise AI Agent Workbench

1. Executive Summary

Стратегическая цель

Enterprise AI Agent Workbench — это модульная платформа для демонстрации production-style AI Engineering: агентные workflow, RAG, Text-to-SQL, MCP/tool use, computer-use, evaluation, observability, governance, incident response и orchestration.

Главная цель проекта — не запустить массовый SaaS, а создать **публичный инженерный артефакт**, который показывает потенциальному работодателю:

- ты понимаешь не только “как вызвать LLM”;
- ты умеешь проектировать **AI-системы вокруг LLM**;
- ты видишь современные проблемы enterprise AI: качество, надёжность, трассировка, безопасность, управление инструментами, контекст, оценка, стоимость, latency;
- ты способен строить не демо-бота, а архитектурно связанный продукт.

Это хорошо совпадает с направлением индустрии: OpenAI описывает агентов как системы, которые самостоятельно выполняют workflow, используют инструменты и работают в рамках guardrails; Anthropic отдельно подчёркивает важность простых composable agentic patterns; McKinsey в 2026 выделяет agentic AI trust, governance и risk controls как ключевые барьеры масштабирования AI в бизнесе. ([OpenAI](#))

North Star Metric

North Star Metric:

Количество завершённых production-grade AI workflow-модулей, у каждого из которых есть: архитектурная схема, рабочее demo, trace выполнения, eval-метрики, README, ограничения и recorded demo.

Для MVP:

3 production-grade модуля + общее ядро платформы.

Для полного portfolio-релиза:

10 модулей на единой Service-Oriented Core архитектуре.

Рекомендация эксперта

Не пытаться за один MVP сделать все 10 модулей одинаково глубоко. Это почти гарантированно приведёт к размытию качества.

Правильная стратегия:

- **MVP:** ядро платформы + 3 самых сильных сценария.
- **Portfolio Release:** все 10 модулей.
- **Showcase Release:** polished UI, видео, документация, архитектурные схемы, публичные посты.

2. Архитектура и стек технологий

Архитектурный принцип

Паттерн: **Service-Oriented Core**

Это означает:

- есть единое ядро платформы;
- каждый AI-модуль подключается как отдельный сервис или app;
- общие возможности не дублируются;
- RAG, evals, observability, MCP, context, policy и orchestration — это не куски внутри каждого проекта, а **общие платформенные сервисы**.

Концептуальная архитектура

Слой	Назначение
UI / Demo Console	Единый интерфейс для запуска сценариев, просмотра traces, evals и результатов
API Gateway	Единая точка входа для UI и внешних клиентов
Workflow Orchestrator	Управление состояниями, шагами, retry, human approval, branching
Agent Runtime	Запуск агентов, tools, memory, instructions, model routing
Context Engine	Управление контекстом, summaries, retrieval, compression, memory
RAG Core	Ingestion, chunking, embeddings, hybrid search, reranking
MCP / Tool Gateway	Подключение инструментов, SQL, browser, файлов, внешних API
Evaluation Engine	RAG evals, agent evals, SQL evals, synthetic datasets

Слой	Назначение
Observability Layer	Traces, logs, token usage, latency, cost, failure taxonomy
Policy / Governance Engine	Guardrails, allow/deny rules, PII checks, approval gates
Demo Apps	10 бизнес-сценариев поверх общего ядра

Современный стек 2026

OpenAI в своих agent-гайдах отдельно выделяет модель, tools и instructions/guardrails как базовые компоненты агента, а также подчёркивает orchestration и evals; Anthropic описывает MCP как открытый стандарт для подключения AI-приложений к внешним системам и данным. Это напрямую ложится в выбранную архитектуру. ([OpenAI](#))

Направление	Рекомендуемый стек	Почему
Frontend	Next.js, React, TypeScript, Tailwind, shadcn/ui	Быстро сделать профессиональный dashboard/demo-console
Backend API	Python, FastAPI, Pydantic, SQLAlchemy	Быстрая разработка AI/backend-сервисов
Agent Runtime	LangGraph или собственная state-machine-обвязка; опционально OpenAI Agents SDK	Для graph/stateful workflows, multi-step agents, tool calling
LLM Gateway	LiteLLM или собственный model-router	Единый слой для локальных моделей и внешних API
Local LLM	Ollama / llama.cpp / vLLM по возможности; Qwen/Llama/Mistral 7B–8B в quantized-режиме	Реалистично для RTX 4070 8 GB
External LLM API	OpenAI / Anthropic / Google как optional hybrid mode	Для сложных reasoning/eval задач, где локальной модели мало
Embeddings	bge-small/bge-base, e5, nomic-embed; локально	Дёшево, быстро, реалистично
Vector DB	Qdrant или pgvector	Qdrant удобен для RAG; pgvector проще в едином PostgreSQL
Relational DB	PostgreSQL	SQL-agent, traces, evals, конфигурации, audit logs
Cache / Queue	Redis	Очереди задач, временное состояние, rate limits
Document Parsing	unstructured, pymupdf, python-docx, pandas/openpyxl	Для документов, PDF, Excel

Направление	Рекомендуемый стек	Почему
Browser / Computer Use	Playwright + screenshot/action traces; optional computer-use API	Для guarded QA-agent по legacy UI
Evaluation	DeepEval, Ragas, promptfoo, custom eval harness	Для RAG/agent/SQL evaluation
Observability	OpenTelemetry, Langfuse, structured logs, custom trace viewer	Для latency, cost, tool calls, errors
Security / Governance	policy engine на YAML/JSON rules, PII checks, allowlist tools, human approval gates	Для enterprise-readiness
DevOps	Docker Compose для local; GitHub Actions; GHCR; optional Terraform	Достаточно для portfolio-grade production style
Cloud	VPS / Hetzner / Fly.io / Render / AWS/GCP/Azure по желанию	Для публичного demo, но MVP можно поднять локально
Mobile	Не делать native mobile в MVP; responsive web/PWA	Mobile не является ключевой ценностью проекта

Важное ограничение RTX 4070 8 GB

Нельзя строить roadmap вокруг тяжёлых локальных LLM уровня 30B/70B без серьёзных компромиссов. Реалистичный подход:

- локально: embeddings, reranker, 7B/8B quantized LLM, простые agents;
- гибридно: сложный reasoning, judge/eval, computer-use — через внешние API;
- архитектура должна позволять переключать модели через **model-router**.

Directory Map

```
enterprise-ai-agent-workbench/  
|  
|— README.md  
|— LICENSE  
|— docker-compose.yml  
|— .env.example  
|— Makefile  
|  
|— docs/  
|   |— 00_product_vision.md  
|   |— 01_architecture.md  
|   |— 02_roadmap.md  
|   |— 03_eval_strategy.md  
|   |— 04_observability.md  
|   |— 05_governance.md  
|   |— 06_demo_scripts.md
```

- └─ diagrams/
 - └─ system_context.mmd
 - └─ service_oriented_core.mmd
 - └─ workflow_orchestrator.mmd
 - └─ module_map.mmd

- └─ platform/

- └─ api-gateway/
 - └─ app/
 - └─ routers/
 - └─ schemas/
 - └─ tests/

- └─ agent-runtime/
 - └─ agents/
 - └─ tools/
 - └─ model_router/
 - └─ prompts/
 - └─ memory/
 - └─ tests/

- └─ workflow-orchestrator/
 - └─ graphs/
 - └─ state/
 - └─ retries/
 - └─ approvals/
 - └─ tests/

- └─ context-engine/
 - └─ context_policies/
 - └─ summarization/
 - └─ compression/
 - └─ memory_retrieval/
 - └─ tests/

- └─ rag-core/
 - └─ ingestion/
 - └─ chunking/
 - └─ embeddings/
 - └─ retrieval/
 - └─ reranking/
 - └─ tests/

- └─ mcp-tool-gateway/
 - └─ servers/
 - └─ clients/
 - └─ tool_registry/
 - └─ permissions/
 - └─ tests/

```
├── eval-engine/
│   ├── datasets/
│   ├── scorers/
│   ├── judges/
│   ├── reports/
│   └── tests/
│
│   ├── observability/
│   │   ├── traces/
│   │   ├── metrics/
│   │   ├── logs/
│   │   ├── cost_tracking/
│   │   └── dashboards/
│   │
│   ├── policy-engine/
│   │   ├── guardrails/
│   │   ├── pii_detection/
│   │   ├── approval_rules/
│   │   ├── tool_allowlists/
│   │   └── audit/
│   │
│   └── shared/
│       ├── config/
│       ├── db/
│       ├── events/
│       ├── schemas/
│       └── utils/
│
├── apps/
│   ├── business-process-investigator/
│   ├── rag-evaluation-lab/
│   ├── text-to-sql-rag-agent/
│   ├── mcp-deep-research-agent/
│   ├── agent-observability-console/
│   ├── compliance-risk-reviewer/
│   ├── guarded-computer-use-qa-agent/
│   ├── incident-response-agent/
│   ├── synthetic-eval-generator/
│   └── enterprise-workflow-orchestrator-demo/
│
├── ui/
│   ├── web/
│   │   ├── app/
│   │   ├── components/
│   │   ├── features/
│   │   ├── lib/
│   │   └── tests/
```

```
├── design-system/
├── examples/
│   ├── sample_documents/
│   ├── sample_sql_database/
│   ├── sample_traces/
│   ├── sample_eval_datasets/
│   ├── sample_legacy_ui/
│   └── demo_workflows/
├── data/
│   ├── raw/
│   ├── processed/
│   ├── vector_store/
│   └── eval_results/
├── infra/
│   ├── docker/
│   ├── github-actions/
│   ├── terraform_optional/
│   └── deployment/
└── tests/
    ├── integration/
    ├── e2e/
    ├── eval_regression/
    └── security/
```

3. Детальная дорожная карта по фазам

Общий план

Релиз	Цель	Реалистичный срок для одного человека
MVP	Ядро платформы + 3 ключевых demo-модуля	8–10 недель
Portfolio Release	Все 10 модулей в рабочем, но не идеальном виде	16–20 недель
Showcase Release	Полировка UI, README, видео, архитектурные схемы, demo scripts	20–24 недели

Рекомендация эксперта: MVP должен включать не все 10 модулей, а те, которые лучше всего демонстрируют архитектурный уровень:

- 1. Text-to-SQL + RAG BI Agent
- 2. Enterprise RAG Evaluation Lab
- 3. AI Workflow Orchestrator
- 4. Минимальный **Observability Layer** как сквозная часть.

Phase 1 — Discovery

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии Инструменты
Discovery 1: Product framing — 2 дня	Зафиксировать цель проекта, аудиторию, финальный список 10 модулей, MVP-scope, non-goals	docs/00_product_vision.md , финальный список модулей, MVP boundary	1 Product Owner / AI Engineer	Markdown, Miro/Excalidraw
Discovery 2: Use-case selection — 2 дня	Выбрать 3 MVP-сценария, определить demo story для каждого	docs/06_demo_scripts.md с 3 сценариями	1 AI Product Architect	Markdown
Discovery 3: Data strategy — 2 дня	Определить демо-данные: документы, SQL schema, traces, synthetic evals	Папка examples/ с планом данных	1 Data/AI Engineer	PostgreSQL CSV, PDF, Markdown
Discovery 4: Risk & feasibility — 1 день	Проверить ограничения RTX 4070 8GB, решить local/API split	docs/feasibility.md	1 AI Engineer	Ollama/llama API providers

Phase 2 — Design

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии
Design 1: System	Спроектировать Service-Oriented	docs/01_architecture.md , Mermaid diagram	1 AI Architect	Mermaid

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Техн
Architecture — 3 дня	Core, API boundaries, основные сервисы			
Design 2: Data & Storage Design — 2 дня	Спроектировать PostgreSQL schema, trace schema, eval schema, document schema	ERD + миграции v0	1 Backend/Data Engineer	Postg
Design 3: Agent Runtime Design — 3 дня	Описать agent contracts, tool contracts, model router, prompts registry	agent-runtime skeleton	1 AI Engineer	Pyda Ager
Design 4: Evaluation Design — 2 дня	Определить eval taxonomy: RAG, SQL, agent, tool, safety	docs/03_eval_strategy.md	1 AI Evaluation Engineer	Raga
Design 5: Observability Design — 2 дня	Определить trace model: run, step, tool call, model call, error, cost	docs/04_observability.md	1 LLMOps Engineer	Open
Design 6: Governance Design — 2 дня	Tool allowlist, approval gates, PII checks, audit log	docs/05_governance.md	1 AI Governance Engineer	YAM

Phase 3 — Development Sprints

Sprint 0 — Foundation

Этап/ Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Sprint 0: Repo & Platform Skeleton	Создать monorepo, Docker Compose, FastAPI skeleton, Next.js UI,	Проект запускается одной командой;	1 Full-stack AI Engineer	Docker, FastAPI, Next.js,

Этап/ Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
— 1 неделя	PostgreSQL, Qdrant/pgvector, Redis	healthcheck работает; README draft		PostgreSQL, Redis

Sprint 1 — Core Agent Runtime

Этап/ Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Sprint 1: Agent Runtime + Model Router — 1 неделя	Реализовать model gateway, prompt registry, tool interface, structured output, basic run loop	Можно запустить простого агента через API и увидеть результат в UI	1 AI/Backend Engineer	FastAPI, Pydantic, LiteLLM/Ollama/API
Sprint 1.1: Context Engine v0	Добавить context builder: system prompt, retrieved chunks, memory, task state	Каждый run сохраняет использованный контекст	1 AI Engineer	Pydantic, PostgreSQL

Sprint 2 — RAG Core + Evaluation Base

Этап/ Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Sprint 2: RAG Core — 1 неделя	Ingestion, chunking, embeddings, vector search, citations	Документы загружаются, индексируются, находятся релевантные chunks	1 AI/Data Engineer	Qdrant/pgvector, PyMuPDF, embeddings
Sprint 2.1: RAG Eval v0	Добавить метрики retrieval hit rate, citation accuracy,	Есть первый eval report по sample documents	1 AI Evaluation Engineer	Ragas/DeepEval/custom

Этап/ Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
	answer groundedness			

OpenAI и Anthropic отдельно выделяют evals как обязательную часть надёжных AI-приложений, особенно для агентов, которые работают многошагово, вызывают tools и меняют состояние. ([OpenAI Developers](#))

Sprint 3 — Text-to-SQL + RAG BI Agent

Этап/ Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Sprint 3: SQL Agent MVP — 1 неделя	Schema explorer, SQL generation, SQL validation, safe query execution, explanation	Пользователь задаёт бизнес-вопрос → получает SQL, результат, объяснение	1 Data/AI Engineer	PostgreSQL, SQLAlchemy, Pydantic
Sprint 3.1: BI UI	Таблица результата, generated SQL, reasoning trace, chart placeholder	UI показывает полный путь: вопрос → SQL → результат	0.5 Frontend + 1 Backend	Next.js, Recharts

Sprint 4 — Workflow Orchestrator

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Sprint 4: Orchestrator v0 — 1 неделя	State machine, steps, transitions, retries, failure states, human approval	Можно собрать workflow из 3–5 шагов и выполнить его	1 AI/Backend Engineer	LangGraph/custom state machine

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Sprint 4.1: Human-in-the-loop	Approval gate перед risky action	UI показывает pending approval и audit log	1 Full-stack Engineer	FastAPI, Next.js, PostgreSQL

Anthropic рекомендует не усложнять агентные системы без необходимости и использовать composable patterns; поэтому оркестратор должен начинаться с предсказуемых workflow, а не с полностью автономной “магии”. (anthropic.com)

Sprint 5 — Observability Platform

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Sprint 5: Trace Logging — 1 неделя	Логировать run, step, prompt, model call, tool call, error, latency, token usage	Каждый агентный запуск имеет полный trace	1 Backend/LLMOps Engineer	OpenTelemetry/custom PostgreSQL
Sprint 5.1: Observability UI	Timeline, tool calls, retrieved chunks, cost, latency, errors	UI показывает “мозг агента” по шагам	1 Frontend Engineer	Next.js, tables, timeline components

Sprint 6 — Business Process Investigator + Compliance Reviewer

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Sprint 6: Process	Документ → сущности → процесс →	На sample T3 система формирует	1 AI Engineer	RAG Core, LLM, Mermaid

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/Инструменты
Investigator — 1 неделя	противоречия → backlog	process map и список задач		
Sprint 6.1: Compliance Reviewer v0	Проверка документа по policy rules, risk scoring, рекомендации	Документ получает risk report с объяснениями	1 AI/Governance Engineer	Policy Engine, RAG, PII checks

Sprint 7 — MCP Deep Research + Tool Gateway

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/Инструменты
Sprint 7: MCP Tool Gateway — 1 неделя	Tool registry, tool permissions, sample MCP-style connectors	Агент вызывает 2–3 инструмента через единый gateway	1 AI/Backend Engineer	MCP SDK / JSON-RPC style contracts
Sprint 7.1: Deep Research Agent	Planner, researcher, verifier, report generator	Агент делает research report с источниками и trace	1 AI Engineer	Agent Runtime, RAG, web/search optional

MCP важен именно как слой стандартизации подключений к данным и tools; но его нужно проектировать с security boundary, allowlist и audit log, потому что tool connectivity увеличивает поверхность риска. (anthropic.com)

Sprint 8 — Guarded Computer-Use QA Agent

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/Инструменты
Sprint 8: Legacy UI Sandbox — 1 неделя	Создать demo legacy web UI: формы, ошибки, сценарии тестирования	Есть локальное legacy-приложение для тестов	1 Full-stack Engineer	Next.js/FastAPI или simple web app
Sprint 8.1: Computer-	Агент смотрит UI, планирует	Агент проходит 2–3 сценария	1 AI/QA Engineer	Playwright, screenshots,

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/Инструменты
Use QA Agent	действия, выполняет через Playwright, делает QA report	и пишет bug report		guardrails

Computer-use — одно из важных направлений для агентов: OpenAI описывает computer-use как возможность модели работать с пользовательским интерфейсом, а Anthropic называет computer use следующим frontier-направлением для моделей, которые взаимодействуют не только через API, но и через UI. ([OpenAI Developers](#))

Sprint 9 — Synthetic Eval Generator + Incident Response

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/Инструменты
Sprint 9: Synthetic Eval Generator — 1 неделя	Генерация вопросов, эталонных ответов, negative cases, multi-hop cases	Из корпуса документов создаётся eval dataset	1 AI Evaluation Engineer	LLM API/local LLM, Pydantic
Sprint 9.1: Incident Response Agent	Агент анализирует failed traces, классифицирует root cause, предлагает fix	Для failed run создаётся RCA report	1 LLMOps Engineer	Observability Layer, Eval Engine

Sprint 10 — Full Integration

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/Инструменты
Sprint 10: Integration & Demo Flows — 1 неделя	Связать все модули через единый UI, привести traces/evals к единому формату	Все 10 модулей доступны из demo console	1 Full-stack AI Engineer	FastAPI, Next.js, Docker

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Sprint 10.1: Documentation Pass	README, architecture, demo scripts, limitations, roadmap	Репозиторий готов к публичному просмотру	1 Technical Writer / AI Engineer	Markdown, Mermaid

Phase 4 — QA

Этап/Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
QA 1: Functional QA — 3 дня	Проверить API, UI, запуск модулей, Docker setup	Все demo scripts проходят локально	1 QA/Engineer	pytest, Playwright
QA 2: Eval Regression — 3 дня	Прогнать evals по RAG, SQL, agent workflows	Есть baseline metrics и regression report	1 AI Evaluation Engineer	Eval Engine
QA 3: Security & Governance QA — 2 дня	Проверить tool allowlist, approval gates, PII checks, prompt injection cases	Есть security checklist и known limitations	1 AI Security/Governance Engineer	Custom tests
QA 4: UX QA — 2 дня	Проверить, понятно ли демо внешнему зрителю	Демо можно понять за 5–7 минут	0.5 Designer / Product Owner	Manual review

Phase 5 — Launch

Этап/ Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Launch 1: Public GitHub Release — 2 дня	Открыть репозиторий, оформить README, добавить screenshots, architecture diagram	GitHub repo public- ready	1 Engineer	GitHub
Launch 2: Demo Deployment — 2 дня	Развернуть демо- версию или записать локальное видео	Есть публичное demo/video	1 DevOps/Engineer	VPS/Render/Fly.io/Dock
Launch 3: Portfolio Packaging — 2 дня	Подготовить summary: что сделано, что показывает, какие trade- offs	Есть landing README и demo guide	1 Product Owner	Markdown, Loom/OBS

Phase 6 — Post-Launch

Этап/ Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Post- Launch 1: Feedback Loop — 2 недели	Собрать обратную связь, Issues, вопросы, улучшения	Backlog v2	1 Product Owner	GitHub Issues
Post- Launch 2: Hardening — 2 недели	Улучшить стабильность, evals, docs, UI	Release v1.1	1 Engineer	CI/CD, tests
Post- Launch 3: Case	Описать 2–3 технических разбора модулей	docs/case_studies/	1 Technical Writer / Engineer	Markdown

Этап/ Спринт	Ключевые задачи	Definition of Done	Ресурсы	Технологии/ Инструменты
Studies — 2 недели				
Post-Launch 4: Public Narrative — ongoing	Серия постов/разборов по архитектуре, evals, observability, computer-use	5–10 технических публикаций	1 Owner	LinkedIn/GitHub/X

4. Финансовый план и ресурсы

Так как проект делается для портфолио, а не как коммерческий SaaS, главный ресурс — **время**. Денежные затраты можно держать низкими.

Основные допущения

Параметр	Допущение
Команда	1 человек: ты как AI Engineer / Product Owner / Backend / DevOps
Бюджет	Не ограничен, но нет смысла искусственно раздувать
Инфраструктура MVP	Локально + optional VPS
LLM	Локальные small models + external API для сложных задач
Данные	Только synthetic/demo data, без реальных конфиденциальных данных
Коммерческий маркетинг	Не нужен на MVP

Оценка затрат по этапам

Этап	ФОТ / трудозатраты	Лицензии/API	Инфраструктура	Маркетинг/ упаковка	К
Discovery	3–7 рабочих дней	\$0–50	\$0	\$0	Тс пр
Design	1–2 недели	\$0–100	\$0	\$0	А с
MVP Development	6–8 недель	\$100–500	\$20–100/мес	\$0–100	А V

Этап	ФОТ / трудоzатраты	Лицензии/API	Инфраструктура	Маркетинг/ упаковка	К
Full Portfolio Release	8–12 дополнительных недель	\$300–1500	\$50–300/мес	\$100–300	Д
Showcase Release	2–4 недели	\$100–500	\$50–300/мес	\$100–500	В
Post-Launch	ongoing	\$50–300/мес	\$50–300/мес	\$0–200/мес	П

Cash Budget

Сценарий	Денежные затраты	Что входит
Lean Local	\$100–500 total	Локальный запуск, минимум API, без постоянного cloud
Public Demo	\$500–2000 total	VPS, API, домен, demo deployment, видео
High Polish Portfolio	\$2000–5000 total	Больше API, хороший UI, платные design/assets, стабильный hosting

Burn Rate после запуска

Категория	Lean	Public Demo
VPS / hosting	\$10–50/мес	\$50–200/мес
LLM API	\$20–150/мес	\$100–500/мес
Observability / logging	\$0–50/мес	\$0–200/мес
Domain / misc	\$1–5/мес	\$1–10/мес
Итого	\$30–250/мес	\$150–900/мес

Рекомендация эксперта

Для твоей цели **не нужен дорогой cloud**. Лучше вложиться не в инфраструктуру, а в:

- качество README;
- архитектурные схемы;
- eval reports;
- demo videos;
- trace viewer;
- понятные ограничения;

- reproducible local setup.

5. Управление рисками

Риск	Тип	Вероятность	Impact	Mitigation Plan
Scope creep из-за 10 модулей	Операционный	High	High	MVP = 3 модуля + ядро; остальные через phased rollout
Проект станет “витриной”, а не инженерной системой	Рыночный/ репутационный	Medium	High	Для каждого модуля обязательны evals, traces, limitations, architecture
RTX 4070 8 GB не потянет тяжёлые модели	Технический	High	Medium	Использовать 7B/8B quantized, API fallback, model-router
Высокая стоимость API при evals	Финансовый	Medium	Medium	Batch evals, caching, маленькие модели для judge там, где возможно
RAG будет выглядеть как обычный “чат с PDF”	Рыночный	Medium	High	Делать акцент на eval lab, corrective RAG, citation accuracy, regression
Text-to-SQL может генерировать опасные SQL	Технический/ безопасность	High	High	Только read-only, SQL parser, allowlist таблиц, LIMIT, sandbox DB
Computer-use агент будет нестабилен	Технический	High	Medium	Работать на controlled legacy UI sandbox, Playwright traces, retries
MCP/tool use создаст security surface	Технический/ безопасность	Medium	High	Tool allowlist, permissions, audit log, no arbitrary shell execution

Риск	Тип	Вероятность	Impact	Mitigation Plan
Недостаточно качественные demo-данные	Операционный	Medium	High	Сделать synthetic enterprise dataset: документы, SQL, traces, policies
Observability займёт слишком много времени	Технический	Medium	Medium	Начать с custom trace schema, не строить полноценный LangSmith-клон
UI будет слабым и испортит впечатление	Репутационный	Medium	Medium	Использовать shadcn/ui, dashboard templates, focus on clarity
Слишком много технологий в одном проекте	Технический	Medium	High	Service-Oriented Core, но минимальный runtime; не внедрять всё сразу
Нет понятного “вау” для зрителя	Рыночный	Medium	High	Каждый модуль должен иметь 2–3 минутный demo script
Сложно поддерживать monorepo	Операционный	Medium	Medium	Чёткая структура, Makefile, Docker Compose, docs
Governance будет выглядеть формально	Репутационный	Medium	Medium	Добавить реальные policy gates: PII, approval, tool permissions, risk report
Synthetic eval dataset будет недостоверным	Технический	Medium	Medium	Human calibration subset, negative cases, benchmark cards
Incident Response Agent будет слабым без реальных инцидентов	Технический	Medium	Medium	Создать synthetic failed traces: bad retrieval, SQL error, timeout, hallucination
Потенциальные работодатели не	Рыночный	Medium	High	Главный README:

Риск	Тип	Вероятность	Impact	Mitigation Plan
поймут масштаб				architecture first, then modules, then demo videos
Проект не будет завершён из-за объёма	Операционный	High	High	Жёсткий release plan: MVP публично через 8–10 недель

6. KPI и метрики успеха

Технические метрики платформы

Категория	Метрика	MVP Target	Portfolio Target
Availability	Demo uptime	Не критично локально	95%+ для публичного demo
Latency	p95 agent response time	< 30 сек для простых workflow	< 60 сек для сложных workflow
RAG Quality	Retrieval hit rate	70%+	85%+
RAG Quality	Citation accuracy	70%+	85%+
SQL Agent	Valid SQL rate	80%+	90%+
SQL Agent	Execution success rate	75%+	90%+
Agent Runtime	Tool call success rate	80%+	90%+
Workflow	Workflow completion rate	70%+	85%+
Observability	Runs with full trace	100%	100%
Governance	Risky actions behind approval gate	100%	100%
Incident Response	RCA generated for failed runs	70%+	90%+
Cost	Cost per demo run	Контролируемо	Отображается в UI
Regression	Eval regression suite pass rate	70%+	90%+

AI Quality Metrics

Направление	Метрики
RAG	context precision, context recall, faithfulness, answer relevance, citation correctness
Text-to-SQL	SQL validity, execution accuracy, schema linking accuracy, business answer correctness
Agents	task completion rate, tool selection accuracy, retry rate, escalation rate
Computer Use	scenario completion, action accuracy, recovery after UI error, bug report usefulness
Governance	blocked unsafe tool calls, PII detection rate, approval compliance
Incident Response	root cause classification accuracy, fix recommendation usefulness

Business / Portfolio Metrics

Так как это не коммерческий SaaS на старте, классические **CAC, LTV, Churn** для MVP не являются главными. Но я всё равно фиксирую их как optional, если проект позже превращать в продукт.

Категория	Метрика	Что означает
Portfolio Signal	GitHub stars / forks	Интерес сообщества
Portfolio Signal	Количество demo views	Смотрят ли проект
Portfolio Signal	Количество технических вопросов от специалистов	Понимают ли глубину
Career Signal	Inbound messages от рекрутеров/техлидов	Главная внешняя карьерная метрика
Career Signal	Количество интервью, где обсуждают проект	Проект работает как самопрезентация
Content Signal	Сохранения/комментарии к постам	Посты не просто лайкают, а изучают
Product Signal	Returning demo users	Люди возвращаются посмотреть подробнее
Commercial Optional	CAC	Сколько стоит привлечь пользователя, если делать SaaS
Commercial Optional	LTV	Потенциальная ценность пользователя, если монетизировать
Commercial Optional	Churn	Отток пользователей, если появится hosted product

Категория	Метрика	Что означает
Commercial Optional	Retention	Возврат пользователей в demo/product

Главные KPI именно для твоей цели

KPI	Target
Публичный GitHub repository с полным README	1
Архитектурные схемы	5+
Production-style demo modules	10
MVP modules высокой глубины	3
Записанные demo videos	3–5
Eval reports	5+
Agent traces в UI	100% запусков
Описанные limitations/failure cases	Для каждого модуля
Технические статьи/посты по проекту	5–10

Итоговая рекомендация

Проект стоит делать как единый Enterprise AI Agent Workbench, но строго модульно.

Не как 10 отдельных репозиториев.

Не как один неуправляемый монолит.

А как одна платформа с Service-Oriented Core и 10 демонстрационными модулями.

Лучший MVP:

1. **Workflow Orchestrator** как ядро.
2. **Text-to-SQL + RAG BI Agent** как самый понятный enterprise use case.
3. **Enterprise RAG Evaluation Lab** как демонстрация зрелого AI Engineering.
4. **Observability Layer** как сквозной proof of production thinking.

После этого добавлять:

5. Business Process Investigator.
6. Compliance & Risk Reviewer.
7. MCP Deep Research Agent.

8. Guarded Computer-Use QA Agent.
9. Synthetic Eval Generator.
10. Incident Response Agent.

Главный принцип: **каждый модуль должен показывать не “модель ответа”, а полный инженерный цикл: context → tools → reasoning → action → eval → trace → governance → demo.**